

1 Question

Write a method that takes a matrix as parameter (2 dimensional array) and returns true if the matrix is a row increasing matrix, false otherwise. A matrix is row increasing if the numbers in each row are in increasing order.

2 Question

Assume the selection sort to sort an array, A , of size n , in ascending order as follows:

```
public static void selectionSort(int A[]) {
    int n = A.length;
    for (int i = 0; i < n-1; i++) {
        int min_idx = i;
        for (int j = i+1; j < n; j++){
            if (A[j] < A[min_idx])
                min_idx = j;
        }
        if (min_idx != i){
            int temp = A[min_idx];
            A[min_idx] = A[i];
            A[i] = temp;
        }
    }
}
```

Write a program that prompts the user to enter the number of websites, the domain names of each websites, and the number of visitors of these websites. Then, the program creates/shows a list that has the websites in *decreasing order of their number of visitors*.

```
Enter number of websites: 3
Enter the name of website: boredpanda.com
Enter the number of visitors: 2255
Enter the name of website: onedio.com
Enter the number of visitors: 3578
Enter the name of website: sosyalon.com
Enter the number of visitors: 1185
Top Websites:
1) onedio.com 3578
2) boredpanda.com 2255
3) sosyalon.com 1185
```

In this program, you have implement sub-operations as methods. You are expected to write *at least 2 methods*.

3 Question

Write a recursive method **increasingDigits**, which takes an integer n as a parameter and returns true if the digits of n are strictly increasing or returns false otherwise.

```
boolean increasingDigits(int n)
```

```
increasingDigits(137) -> true
```

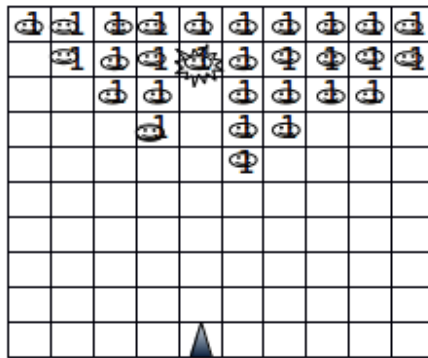
```
increasingDigits(1344567) -> false
```

```
increasingDigits(12345671) -> false
```

4 Question

Suppose that a spaceship game area is modeled by a two dimensional 100 by 100 integer array. In this matrix, enemies are represented by value 1 and empty cells are represented by value 0. The spaceship can be at any position and hit the enemies to destroy them. The direction of the spaceship is always up (north) and the spaceship can hit an enemy if both of them are on the same column. If there are more than one enemies in front of the spaceship on the column, only the closest enemy will be destroyed. If an enemy is hit, its corresponding value at the array must be turned into 0.

Write a method named **hitEnemy**, which takes a two dimensional array (game area) and the position of the spaceship with its column and row indices (cx,cy) as arguments to simulate spaceship fire. An example matrix representing a 10x10 game area is shown below.



1	1	1	1	1	1	1	1	1	1
0	1	1	1	1	1	1	1	0	0
0	0	1	1	0	1	0	0	0	0
0	0	1	1	0	1	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	.	.	.						
.									
.									
.									
				.					

The **hitEnemy** method checks whether the fire from position cx-cy hits an enemy or not. When it hits the first enemy, it removes the enemy (turns 1 into 0) from the game matrix and returns true. If there is no enemy to hit on that column, it returns false.

5 Question

Let say you have the function **primes**

```
public static int[] primes(int N)
```

which returns all primes $\leq N$. Write a function **printCoprimes** which takes two integers as input M, N and will print the co-prime factors of those two integers. The co-prime factors of two integers are the prime numbers who divide both integers. You must use the **primes** function given above. Write also the function **primes**.

```
public static void printCoprimes(int M, int N)
```

```
M = 120, N = 72
```

```
2
```

```
3
```

```
M = 11, N = 121
```

```
11
```

```
M = 2310, N = 30
```

```
2
```

```
3
```

```
5
```

6 Question

Write a function **sort**, which takes a 2d array **A** as arguments and sorts the array. In order to write the function, first you will create a one dimensional array from **A**, then sort the one dimensional array, then convert the sorted one dimensional array to 2d-array and return it. Declare and write also two helper methods for this problem.

$$A =$$

3	7	12	1
1	8	4	6
6	14	0	2

$$A' =$$

0	1	1	2
3	4	6	6
7	8	12	14